

Izrada stilizirane Product Card aplikacije

U ovoj vježbi izradit ćemo malu React aplikaciju koja prikazuje karticu proizvoda. Aplikacija će koristiti tri različita načina stiliziranja:

1. **Plain CSS** za globalni layout aplikacije.
 2. **CSS Modules** za stiliziranje `ProductCard` komponente.
 3. **Inline styles** za dinamičke stilove koji ovise o propsima ili stateu.
-

Cilj vježbe

Cilj je da učenici razumiju kada koristiti koji pristup stiliziranju.

Učenici će vježbati:

- importanje obične CSS datoteke
 - korištenje globalnih CSS klasa
 - izradu `.module.css` datoteke
 - importanje CSS Module objekta
 - korištenje `className={styles.someClass}`
 - korištenje inline stilova za dinamičke vrijednosti
 - uvjetno stiliziranje na temelju podataka
 - strukturiranje UI-ja kroz komponente
-

Naziv projekta

08_React/10_Practice_ProductCardStyling/

Scenarij

Zamislite da izrađujete malu stranicu za prikaz proizvoda u online trgovini.

Stranica treba prikazati jednu karticu proizvoda. Kartica treba sadržavati:

Product name
Category
Price

Stock status
Short description
Button

Ako je proizvod dostupan, status treba biti zelen i gumb treba pisati:

Add to Cart

Ako proizvod nije dostupan, status treba biti crven i gumb treba pisati:

Out of Stock

Dio stilova treba biti globalan, dio lokaliziran samo na komponentu, a dio izračunat direktno u JSX-u.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest product-card-styling -- --template react
```

Zatim uđite u projekt:

```
cd product-card-styling
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Očistite početni projekt

U `src/App.jsx` obrišite početni sadržaj i pripremite osnovnu strukturu:

```
import './App.css';
```

```
function App() {  
  return (  
    <main className="app">
```

```
    <h1>Product Showcase</h1>
  </main>
);
}
```

export default App;

U `src/App.css` obrišite postojeći CSS.

Dio 1 — Plain CSS za globalni layout

3. Stilizirajte osnovni layout pomoću običnog CSS-a

U `src/App.css` dodajte globalne stilove za cijelu aplikaciju.

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 900px;
  margin: 50px auto;
  padding: 32px;
}

.app h1 {
  text-align: center;
  margin-bottom: 32px;
}
```

Ovdje koristimo obični CSS jer se radi o globalnom layoutu aplikacije. Stilovi za `body`, `.app` i naslov nisu specifični samo za jednu malu komponentu.

Dio 2 — CSS Modules za komponentu

4. Napravite `ProductCard` komponentu

U folderu **src** napravite novu datoteku:

ProductCard.jsx

Dodajte početni kod:

```
function ProductCard() {  
  return (  
    <article>  
      <h2>Wireless Headphones</h2>  
      <p>Electronics</p>  
      <p>€89.99</p>  
      <p>High-quality wireless headphones with noise cancellation.</p>  
      <button>Add to Cart</button>  
    </article>  
  );  
}
```

```
export default ProductCard;
```

Zatim je prikažite u **App.jsx**:

```
import ProductCard from "../ProductCard";  
import "../App.css";
```

```
function App() {  
  return (  
    <main className="app">  
      <h1>Product Showcase</h1>  
      <ProductCard />  
    </main>  
  );  
}
```

```
export default App;
```

5. Napravite CSS Module datoteku

U folderu **src** napravite datoteku:

ProductCard.module.css

Dodajte stilove:

```
.card {  
  max-width: 420px;  
  margin: 0 auto;
```

```
padding: 24px;
background-color: white;
border: 1px solid #d8dee9;
border-radius: 16px;
}

.title {
margin-top: 0;
margin-bottom: 8px;
font-size: 28px;
}

.category {
margin: 0 0 16px;
color: #667085;
font-size: 14px;
text-transform: uppercase;
letter-spacing: 0.08em;
}

.price {
margin: 0 0 16px;
font-size: 24px;
font-weight: bold;
}

.description {
margin-bottom: 20px;
line-height: 1.6;
color: #344054;
}

.button {
padding: 10px 16px;
border: 1px solid #ccd3df;
border-radius: 8px;
background-color: #f7f9fc;
cursor: pointer;
font-size: 15px;
}

.button:hover {
background-color: #e9eef6;
}
```

6. Importajte CSS Module u komponentu

U `ProductCard.jsx` dodajte import:

```
import styles from "../ProductCard.module.css";
```

Zatim primijenite klase:

```
import styles from "../ProductCard.module.css";

function ProductCard() {
  return (
    <article className={styles.card}>
      <h2 className={styles.title}>Wireless Headphones</h2>
      <p className={styles.category}>Electronics</p>
      <p className={styles.price}>€89.99</p>
      <p className={styles.description}>
        High-quality wireless headphones with noise cancellation.
      </p>
      <button className={styles.button}>Add to Cart</button>
    </article>
  );
}
```

```
export default ProductCard;
```

Ovdje učenici trebaju primijetiti da ne pišemo:

```
className="card"
```

nego:

```
className={styles.card}
```

To znači da klasa dolazi iz CSS Module objekta i bit će lokalizirana na ovu komponentu.

Dio 3 — Props i dinamički podaci

7. Dodajte props u `ProductCard`

Sada promijenite komponentu tako da prima podatke kroz props:

```
import styles from "../ProductCard.module.css";
```

```
function ProductCard({
  name,
  category,
```

```

    price,
    description,
    inStock
  }) {
    return (
      <article className={styles.card}>
        <h2 className={styles.title}>{name}</h2>
        <p className={styles.category}>{category}</p>
        <p className={styles.price}>€{price}</p>
        <p className={styles.description}>{description}</p>
        <button className={styles.button}>
          {inStock ? "Add to Cart" : "Out of Stock"}
        </button>
      </article>
    );
  }

```

```
export default ProductCard;
```

U `App.jsx` proslijedite podatke:

```

import ProductCard from "../ProductCard";
import "../App.css";

```

```

function App() {
  return (
    <main className="app">
      <h1>Product Showcase</h1>

      <ProductCard
        name="Wireless Headphones"
        category="Electronics"
        price={89.99}
        description="High-quality wireless headphones with noise cancellation."
        inStock={true}
      />
    </main>
  );
}

```

```
export default App;
```

Dio 4 — Inline styles za dinamičko stiliziranje

8. Dodajte status proizvoda

U `ProductCard.jsx` dodajte status poruku prije gumba.

Ako je proizvod dostupan, prikažite:

In stock

Ako nije dostupan, prikažite:

Out of stock

Dodajte inline style koji mijenja boju na temelju `inStock` vrijednosti:

```
const statusStyle = {  
  color: inStock ? "#176b3a" : "#c0392b",  
  fontWeight: "bold"  
};
```

Zatim u JSX dodajte:

```
<p style={statusStyle}>  
  {inStock ? "In stock" : "Out of stock"}  
</p>
```

Cijela komponenta sada izgleda ovako:

```
import styles from "../ProductCard.module.css";
```

```
function ProductCard({  
  name,  
  category,  
  price,  
  description,  
  inStock  
) {  
  const statusStyle = {  
    color: inStock ? "#176b3a" : "#c0392b",  
    fontWeight: "bold"  
  };  
  
  return (  
    <article className={styles.card}>  
      <h2 className={styles.title}>{name}</h2>  
      <p className={styles.category}>{category}</p>  
      <p className={styles.price}>€{price}</p>  
      <p className={styles.description}>{description}</p>  
  
      <p style={statusStyle}>
```



```

    {inStock ? "In stock" : "Out of stock"}
  </p>

  <button className={styles.button}>
    {inStock ? "Add to Cart" : "Out of Stock"}
  </button>
</article>
);
}

```

export default ProductCard;

Ovdje koristimo inline style zato što boja ovisi o podatku `inStock`.

To je dobar primjer situacije u kojoj inline stilovi imaju smisla: kada se neka vrijednost izračunava iz propsa ili statea.

Dio 5 — Uvjetna klasa za button

9. Dodajte disabled stil za gumb

U `ProductCard.module.css` dodajte novu klasu:

```

.disabledButton {
  opacity: 0.5;
  cursor: not-allowed;
}

```

Sada u `ProductCard.jsx` promijenite button:

```

<button
  className={` ${styles.button} ${!inStock ? styles.disabledButton : ""}`}
  disabled={!inStock}
>
  {inStock ? "Add to Cart" : "Out of Stock"}
</button>

```

Ovdje kombiniramo dvije CSS Module klase.

Ako je `inStock true`, gumb ima samo osnovnu klasu:

`styles.button`

Ako je `inStock false`, dobiva i dodatnu klasu:

Očekivano ponašanje aplikacije

Aplikacija treba prikazati karticu proizvoda.

Ako je `inStock={true}`:

Status je zelen.

Gumb je aktivan.

Tekst gumba je Add to Cart.

Ako promijenite `inStock={false}`:

Status je crven.

Gumb je disabled.

Tekst gumba je Out of Stock.

Puno rješenje

`src/App.jsx`

```
import ProductCard from "../ProductCard";
import "../App.css";
```

```
function App() {
  return (
    <main className="app">
      <h1>Product Showcase</h1>

      <ProductCard
        name="Wireless Headphones"
        category="Electronics"
        price={89.99}
        description="High-quality wireless headphones with noise cancellation."
        inStock={true}
      />
    </main>
  );
}
```

```
export default App;
```

src/ProductCard.jsx

```
import styles from "../ProductCard.module.css";

function ProductCard({
  name,
  category,
  price,
  description,
  inStock
}) {
  const statusStyle = {
    color: inStock ? "#176b3a" : "#c0392b",
    fontWeight: "bold"
  };

  return (
    <article className={styles.card}>
      <h2 className={styles.title}>{name}</h2>

      <p className={styles.category}>{category}</p>

      <p className={styles.price}>€{price}</p>

      <p className={styles.description}>{description}</p>

      <p style={statusStyle}>
        {inStock ? "In stock" : "Out of stock"}
      </p>

      <button
        className={` ${styles.button} ${!inStock ? styles.disabledButton : ""}`}
        disabled={!inStock}
      >
        {inStock ? "Add to Cart" : "Out of Stock"}
      </button>
    </article>
  );
}

export default ProductCard;
```

src/ProductCard.module.css

```
.card {  
  max-width: 420px;  
  margin: 0 auto;  
  padding: 24px;  
  background-color: white;  
  border: 1px solid #d8dee9;  
  border-radius: 16px;  
}
```

```
.title {  
  margin-top: 0;  
  margin-bottom: 8px;  
  font-size: 28px;  
}
```

```
.category {  
  margin: 0 0 16px;  
  color: #667085;  
  font-size: 14px;  
  text-transform: uppercase;  
  letter-spacing: 0.08em;  
}
```

```
.price {  
  margin: 0 0 16px;  
  font-size: 24px;  
  font-weight: bold;  
}
```

```
.description {  
  margin-bottom: 20px;  
  line-height: 1.6;  
  color: #344054;  
}
```

```
.button {  
  padding: 10px 16px;  
  border: 1px solid #ccd3df;  
  border-radius: 8px;  
  background-color: #f7f9fc;  
  cursor: pointer;  
  font-size: 15px;  
}
```

```
.button:hover {  
  background-color: #e9eef6;  
}
```

```
.disabledButton {  
  opacity: 0.5;  
  cursor: not-allowed;  
}
```

src/App.css

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  background-color: #eef2f7;  
}
```

```
.app {  
  max-width: 900px;  
  margin: 50px auto;  
  padding: 32px;  
}
```

```
.app h1 {  
  text-align: center;  
  margin-bottom: 32px;  
}
```

Dodatni izazovi

Izazov 1: Dodajte još proizvoda

U `App.jsx` prikazite tri `ProductCard` komponente s različitim podacima.

Primjer:

```
<ProductCard  
  name="Mechanical Keyboard"  
  category="Accessories"  
  price={129.99}  
  description="A compact mechanical keyboard with tactile switches."  
  inStock={false}  
>
```

Nakon toga dodajte globalni layout za kartice.

U `App.jsx` omotajte kartice u `section`:

```
<section className="product-grid">
  {/* ProductCard komponente */}
</section>
```

U `App.css` dodajte:

```
.product-grid {
  display: flex;
  gap: 20px;
  justify-content: center;
  flex-wrap: wrap;
}
```

Izazov 2: Dodajte discount badge

Dodajte novi prop:

```
discount={20}
```

Ako proizvod ima discount veći od `0`, prikažite badge:

20% OFF

Za badge koristite CSS Module klasu.

Primjer:

```
.badge {
  display: inline-block;
  margin-bottom: 12px;
  padding: 4px 8px;
  border-radius: 999px;
  background-color: #fff4cc;
  color: #8a5a00;
  font-size: 13px;
  font-weight: bold;
}
```

U JSX-u:

```
{discount > 0 && (
  <span className={styles.badge}>{discount}% OFF</span>
)}
```

Izazov 3: Dinamička boja cijene

Ako je proizvod na popustu, cijena treba biti drugačije boje.

Koristite inline style:

```
const priceStyle = {  
  color: discount > 0 ? "#c0392b" : "#101828"  
};
```

Zatim:

```
<p className={styles.price} style={priceStyle}>  
  €{price}  
</p>
```

Ovdje se inline style koristi zato što vrijednost ovisi o `discount` propu.

Izazov 4: Usporedite plain CSS i CSS Modules

Dodajte u `App.css` globalnu klasu:

```
.title {  
  color: red;  
}
```

Zatim provjerite utječe li ona na:

```
<h2 className={styles.title}>{name}</h2>
```

Učenici bi trebali primijetiti da globalna `.title` klasa ne mijenja CSS Module klasu, jer `styles.title` dobiva automatski generirano ime klase.